

Subject: Python for Data Science(BTCO13602)
OEP Title: Banking Fraud Detection System
Members: Rudri Desai, Krisha Dhola, Diya Patel, Kavya Patel, Masoom Parekh
Enrollment No:
ET23BTCO008,ET23BTCO009,E23BTCO010,ET23BTCO016,ET23BTCO031

Banking Fraud Detection System

Data:

```
import pandas as pd
data=pd.read_csv("bank_transactions_final.csv")
df=pd.DataFrame(data)
df
```

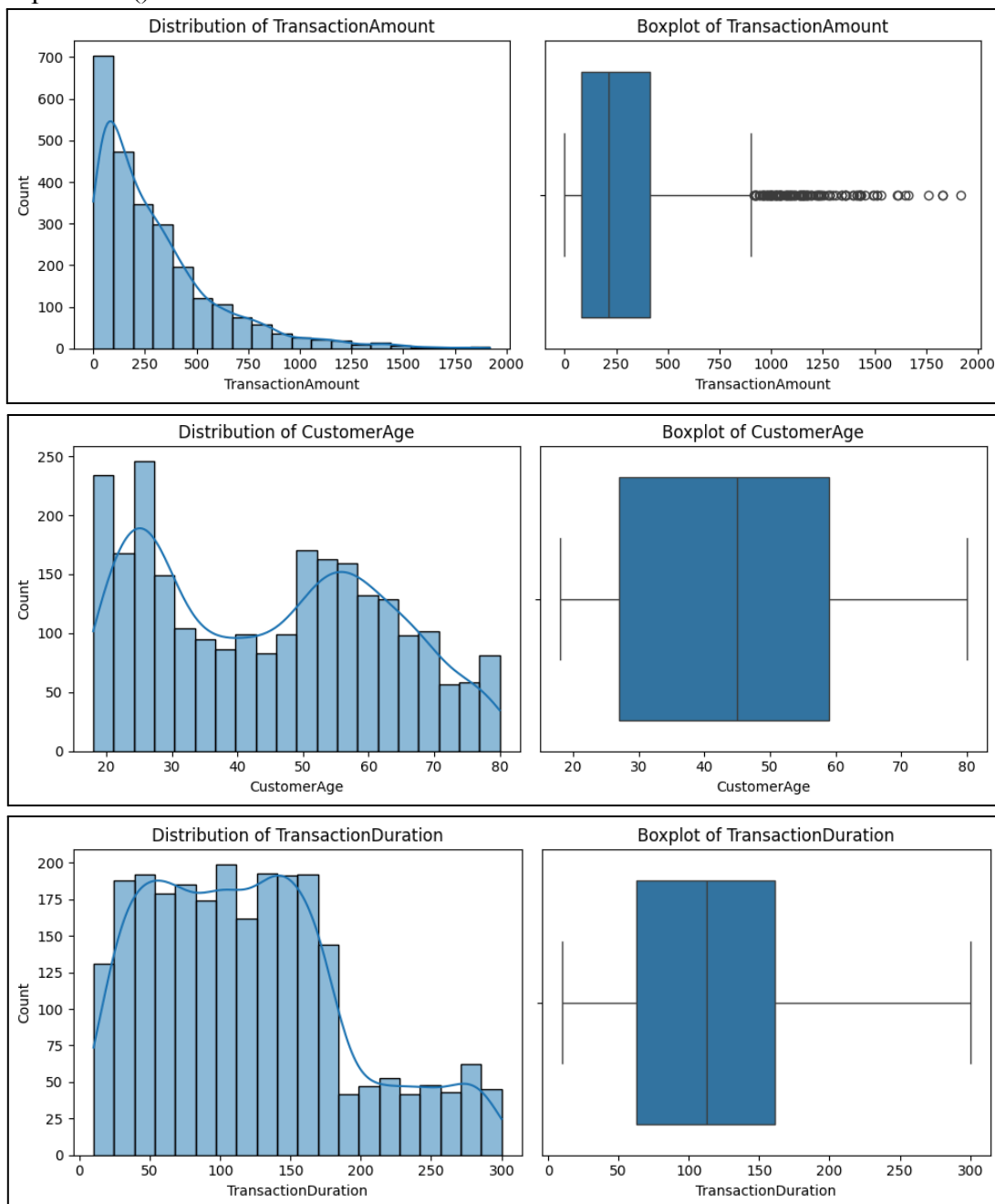
	TransactionID	AccountID	TransactionAmount	TransactionDate	TransactionType	Location	DeviceID	IP Address	MerchantID	Channel	CustomerAge	CustomerOccupation	TransactionDu
0	TX000001	AC00128	14.09	2023-04-11 16:29:14	Debit	San Diego	D000380	162.198.218.92	M015	ATM	70	Doctor	
1	TX000002	AC00455	376.24	2023-06-27 16:44:19	Debit	Houston	D000051	13.149.61.4	M052	ATM	68	Doctor	
2	TX000003	AC00019	126.29	2023-07-10 18:16:08	Debit	Mesa	D000235	215.97.143.157	M009	Online	19	Student	
3	TX000004	AC00070	184.50	2023-05-05 16:32:11	Debit	Raleigh	D000187	200.13.225.150	M002	Online	26	Student	
4	TX000005	AC00411	13.45	2023-10-16 17:51:24	Credit	Atlanta	D000308	65.164.3.100	M091	Online	26	Student	
...	...	...	...	...	...	...	...	...	...	...	...	...	...
2507	TX002508	AC00297	856.21	2023-04-26 17:09:36	Credit	Colorado Springs	D000625	21.157.41.17	M072	Branch	33	Doctor	
2508	TX002509	AC00322	251.54	2023-03-22 17:36:48	Debit	Tucson	D000410	49.174.157.140	M029	Branch	48	Doctor	
2509	TX002510	AC00095	28.63	2023-08-21 17:08:50	Debit	San Diego	D000095	58.1.27.124	M087	Branch	56	Retired	
2510	TX002511	AC00118	185.97	2023-02-24 16:24:46	Debit	Denver	D000634	21.190.11.223	M041	Online	23	Student	
2511	TX002512	AC00009	243.08	2023-02-14 16:21:23	Credit	Jacksonville	D000215	59.127.135.25	M041	Online	24	Student	

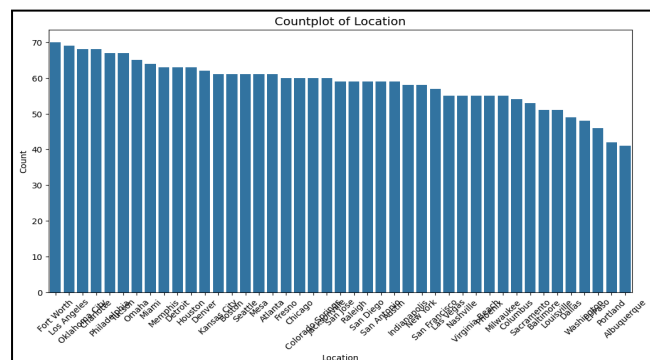
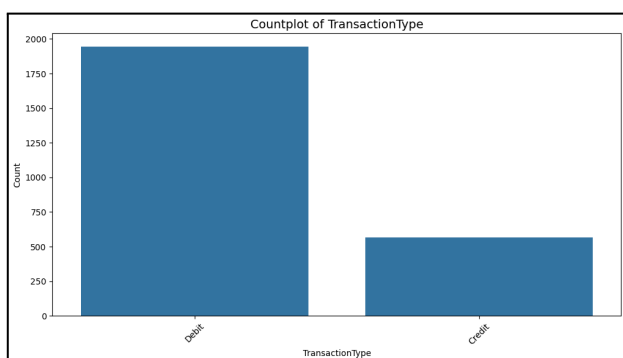
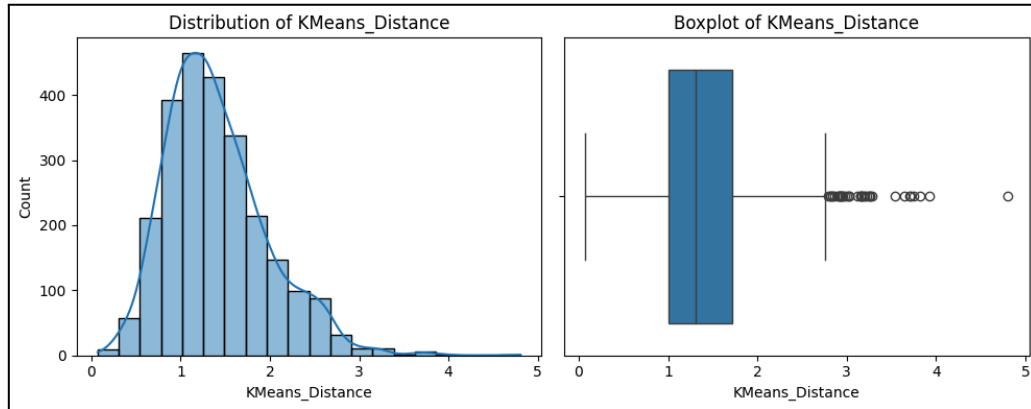
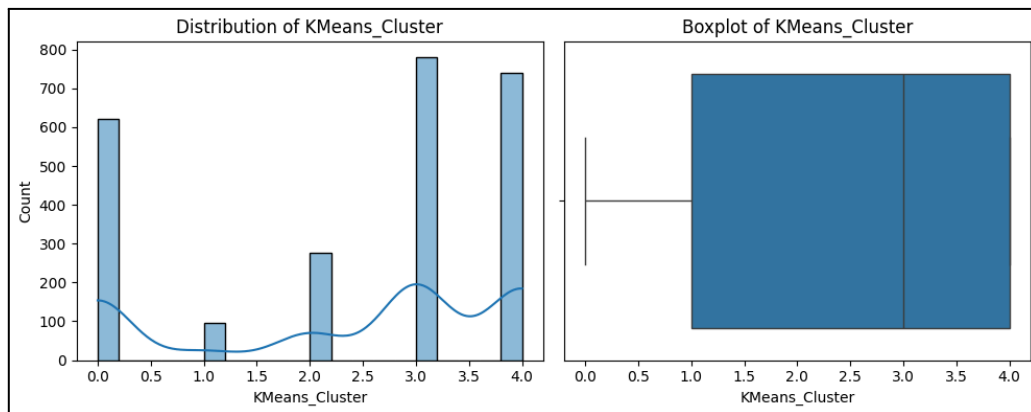
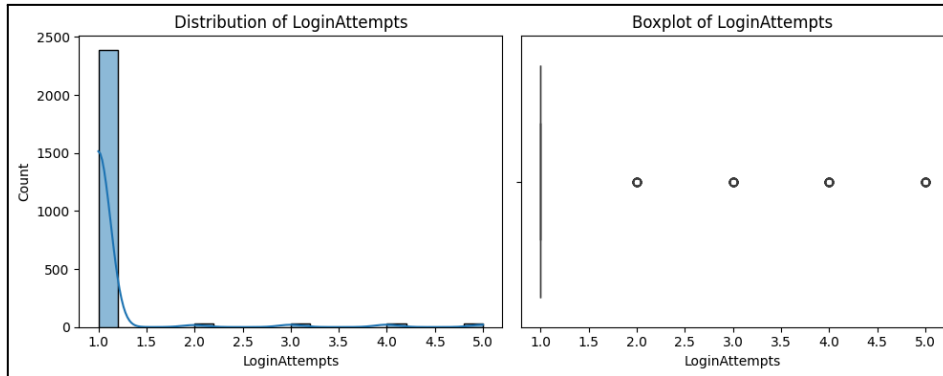
Univariate analysis:

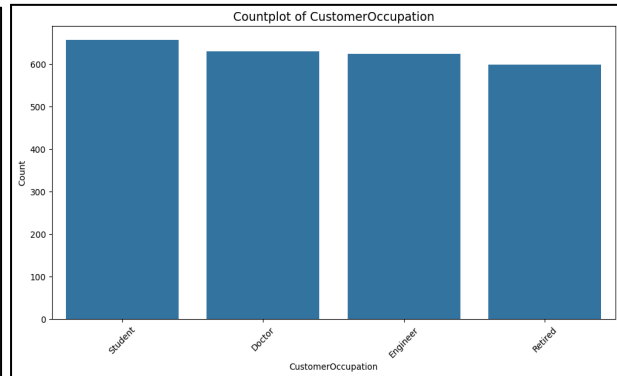
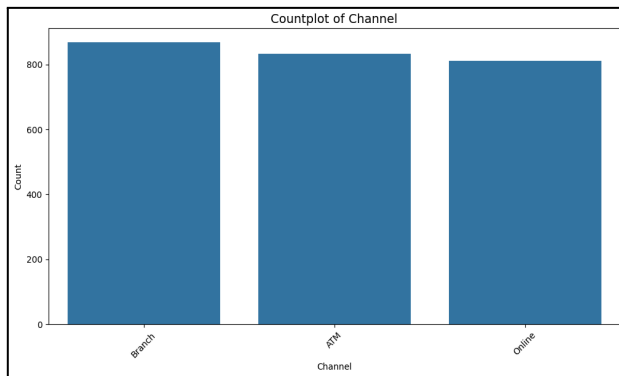
```
import matplotlib.pyplot as plt
import seaborn as sns
# Identify column types
numeric_cols = data.select_dtypes(include=['int64', 'float64']).columns
categorical_cols = data.select_dtypes(include=['object', 'category']).columns
# =====
# NUMERIC VARIABLES
# =====
for col in numeric_cols:
    plt.figure(figsize=(10, 4))
    # Histogram
    plt.subplot(1, 2, 1)
    sns.histplot(data[col], kde=True, bins=20)
    plt.title(f'Distribution of {col}')
    # Boxplot
    plt.subplot(1, 2, 2)
    sns.boxplot(x=data[col])
    plt.title(f'Boxplot of {col}')
    plt.tight_layout()
    plt.show()
categorical_cols = ['TransactionType', 'Location', 'Channel', 'CustomerOccupation']
for col in categorical_cols:
    plt.figure(figsize=(12, 6))
```

```
sns.countplot(  
    x=data[col],  
    order=data[col].value_counts().index  
)
```

```
plt.title(f'Countplot of {col}', fontsize=14)  
plt.xlabel(col)  
plt.ylabel('Count')  
plt.xticks(rotation=45)  
plt.show()
```

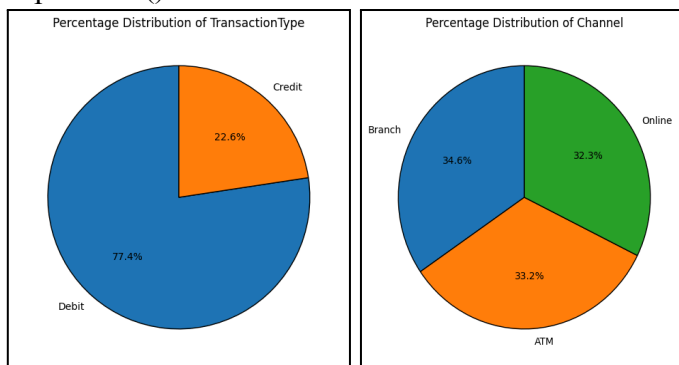






Piecharts:

```
import matplotlib.pyplot as plt
# Relevant columns for pie chart
pie_cols = ['TransactionType', 'Channel']
for col in pie_cols:
    plt.figure(figsize=(6, 6))
    data[col].value_counts().plot.pie(
        autopct='%1.1f%%', # show percentage
        startangle=90,
        wedgeprops={'edgecolor': 'black'})
    plt.title(f'Percentage Distribution of {col}')
    plt.ylabel("") # remove default ylabel
    plt.show()
```

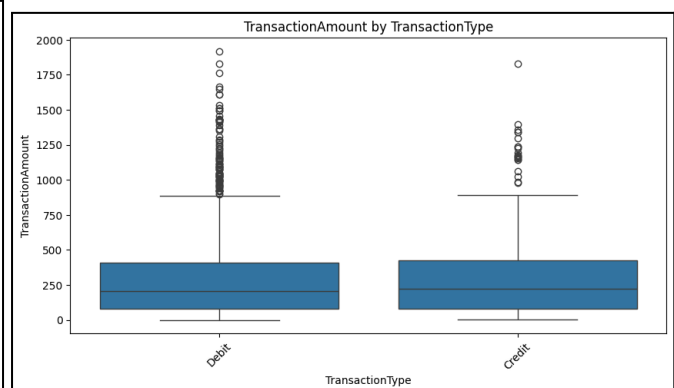
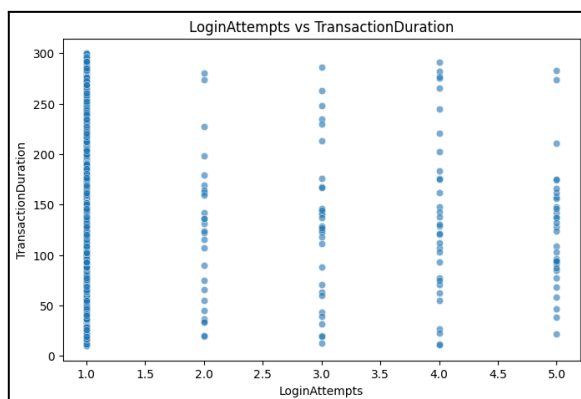
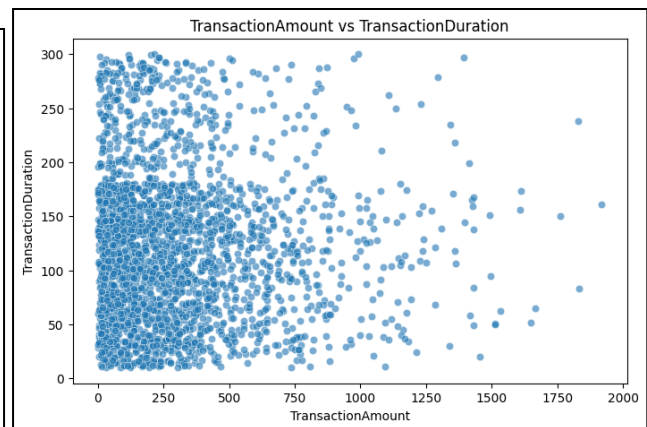
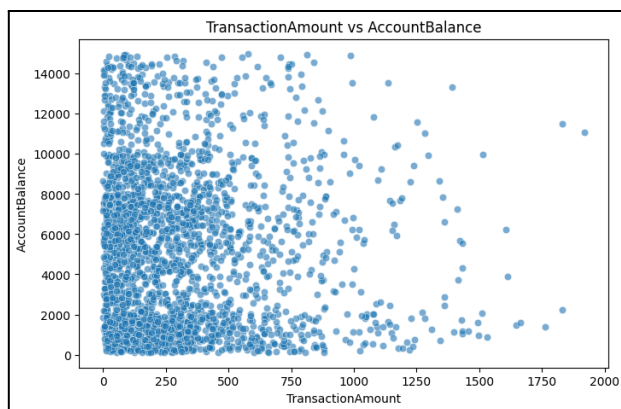
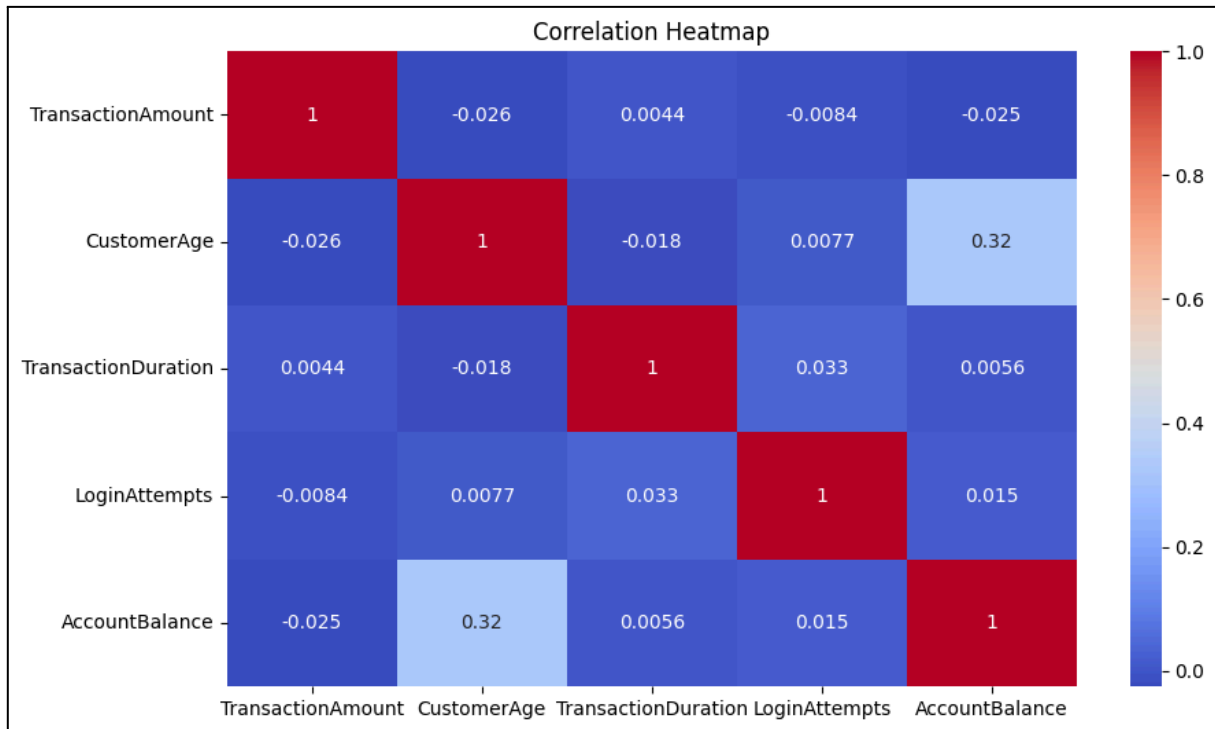


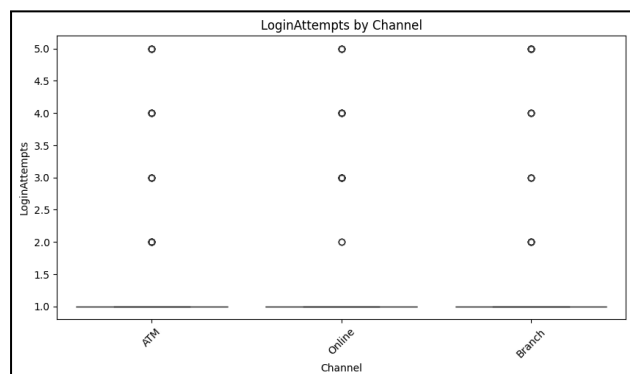
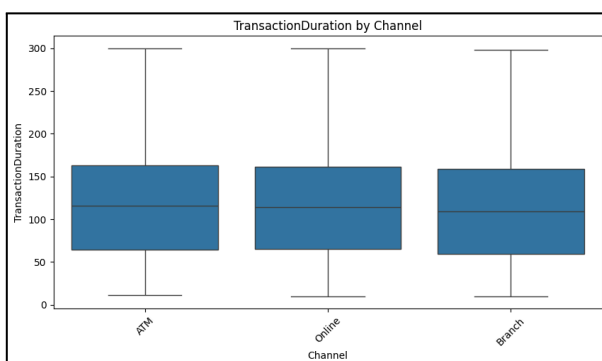
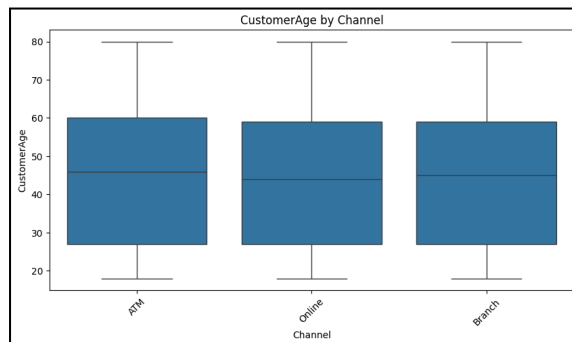
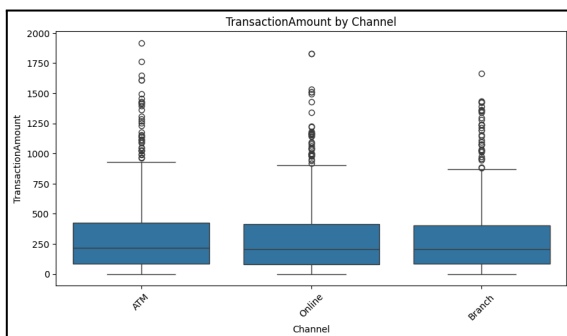
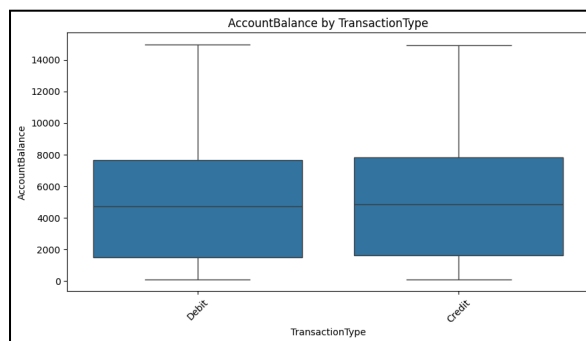
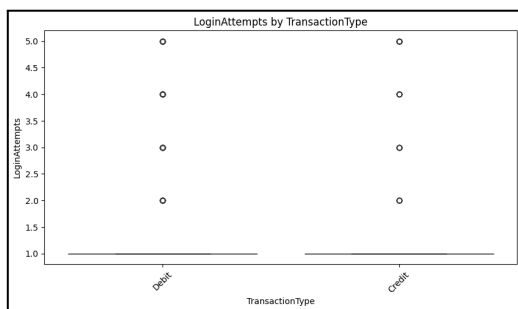
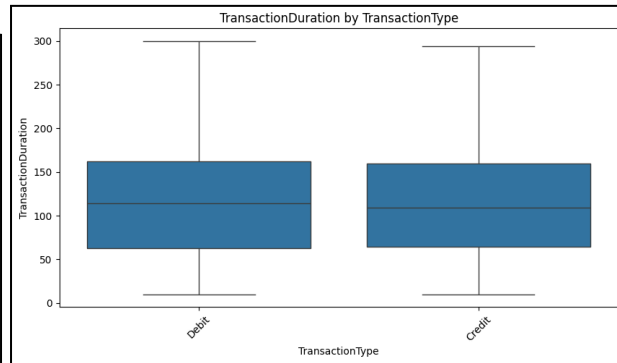
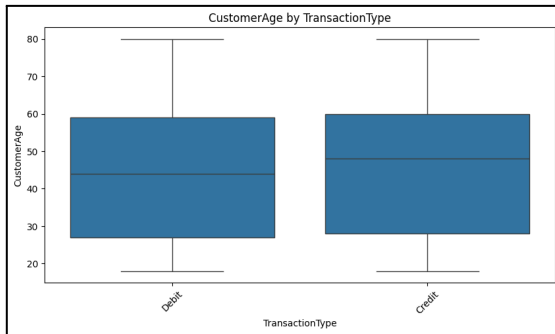
Bivariate analysis:

```
# Define columns
numeric_cols = ['TransactionAmount', 'CustomerAge', 'TransactionDuration', 'LoginAttempts',
'AccountBalance']
categorical_cols = ['TransactionType', 'Location', 'Channel', 'CustomerOccupation']
# =====
# ♦ 1. CORRELATION HEATMAP
# =====
plt.figure(figsize=(10, 6))
sns.heatmap(data[numeric_cols].corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

Subject: Python for Data Science(BTCO13602)
OEP Title: Banking Fraud Detection System
Members:.,Rudri Desai, Krisha Dhola, Diya Patel, Kavya Patel, Masoom Parekh
Enrollment No:
ET23BTCO008,ET23BTCO009,E23BTCO010,ET23BTCO016,ET23BTCO031

```
# =====  
# ♦ 2. IMPORTANT SCATTERPLOTS (SELECTIVE)  
# =====  
important_pairs = [  
    ('TransactionAmount', 'AccountBalance'),  
    ('TransactionAmount', 'TransactionDuration'),  
    ('LoginAttempts', 'TransactionDuration')  
]  
for x, y in important_pairs:  
    plt.figure(figsize=(8, 5))  
    sns.scatterplot(x=data[x], y=data[y], alpha=0.6)  
    plt.title(f'{x} vs {y}')  
    plt.show()  
# =====  
# ♦ 3. BOXPLOTS (NUMERIC vs CATEGORICAL)  
# =====  
important_cat = ['TransactionType', 'Channel']  
  
for cat in important_cat:  
    for num in numeric_cols:  
        plt.figure(figsize=(10, 5))  
        sns.boxplot(x=cat, y=num, data=data)  
        plt.title(f'{num} by {cat}')  
        plt.xticks(rotation=45)  
        plt.show()  
# =====  
# ♦ 4. CATEGORICAL vs CATEGORICAL (LIMITED)  
# =====  
# Channel vs TransactionType  
plt.figure(figsize=(10, 6))  
sns.countplot(x='Channel', hue='TransactionType', data=data)  
plt.title('Channel vs Transaction Type')  
plt.show()
```





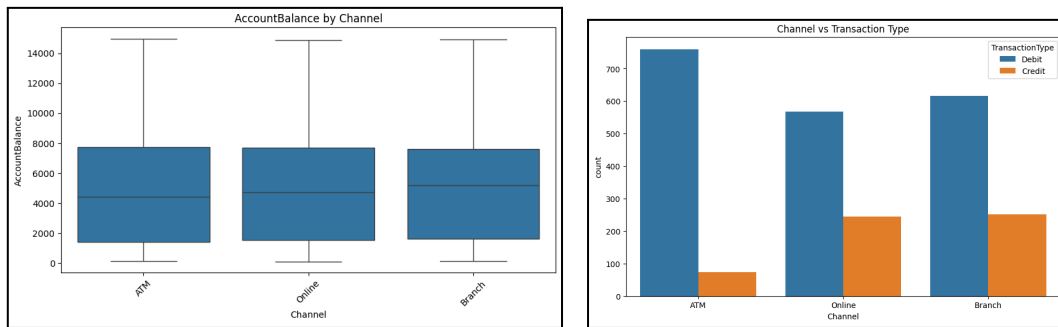
Subject: Python for Data Science(BTCO13602)

OEP Title: Banking Fraud Detection System

Members:.,Rudri Desai, Krisha Dhola, Diya Patel, Kavya Patel, Masoom Parekh

Enrollment No:

ET23BTCO008,ET23BTCO009,E23BTCO010,ET23BTCO016,ET23BTCO031



Multivariate Analysis:

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
selected_numeric = ['TransactionAmount', 'AccountBalance', 'CustomerAge', 'TransactionDuration',  
'LoginAttempts']
```

```
sns.pairplot(  
    data,  
    vars=selected_numeric,  
    hue='TransactionType',  
    corner=True  
)
```

```
plt.suptitle('Pairplot of Key Numerical Features by Transaction Type', y=1.02)
```

```
plt.show()
```

```
g = sns.FacetGrid(  
    data,  
    col="Channel",  
    hue="TransactionType",  
    height=4,  
    aspect=1.2  
)
```

```
g.map(sns.scatterplot, "TransactionAmount", "AccountBalance", alpha=0.7)
```

```
g.add_legend()
```

```
g.fig.suptitle(  
    'Transaction Amount vs Account Balance across Channels',  
    y=1.05  
)
```

```
plt.show()
```

```
transaction_types = ['debit', 'credit']
```

```
for t_type in transaction_types:
```

```
    subset = data[data["TransactionType"].str.lower() == t_type]
```

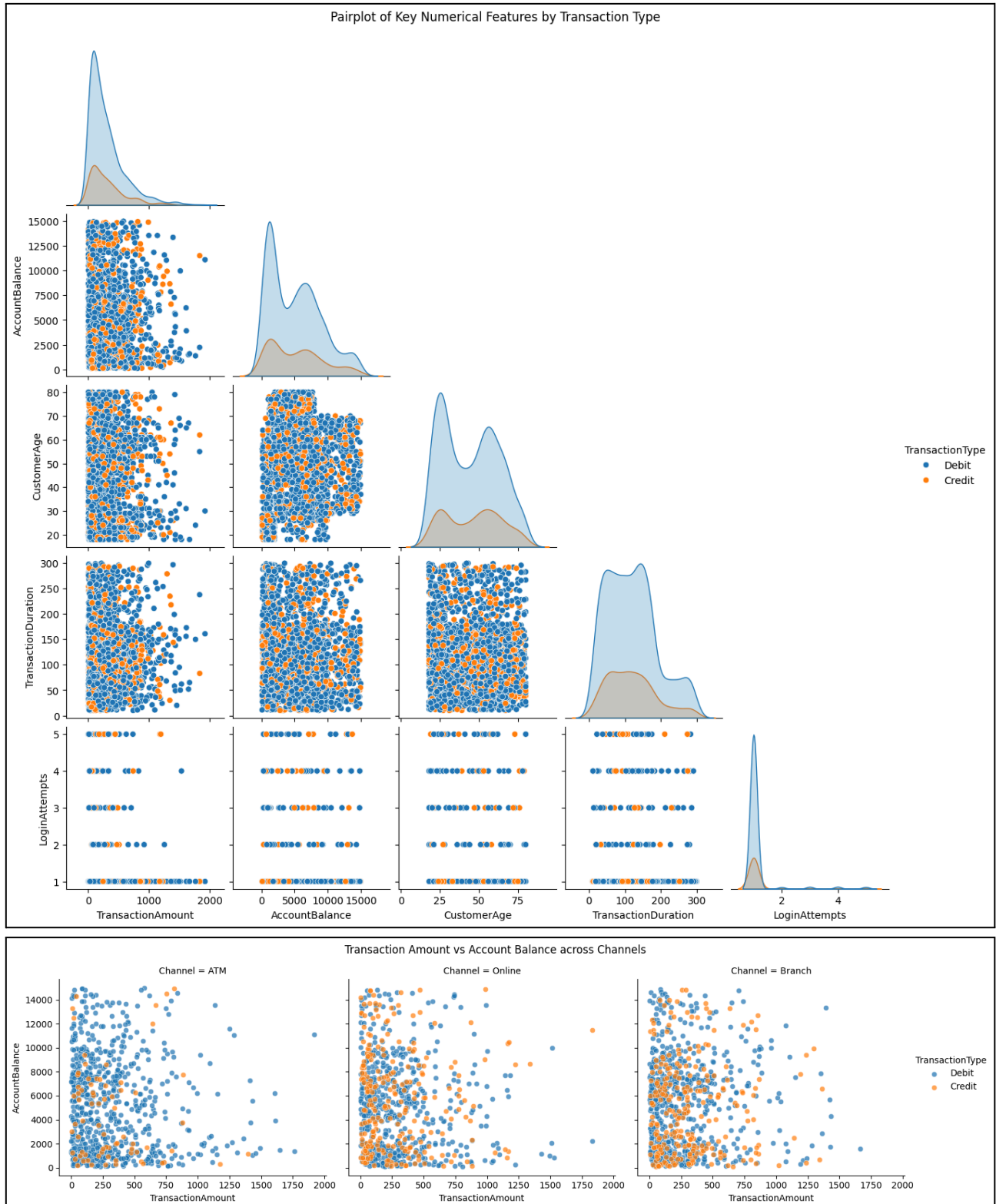
```
plt.figure(figsize=(8, 6))
```

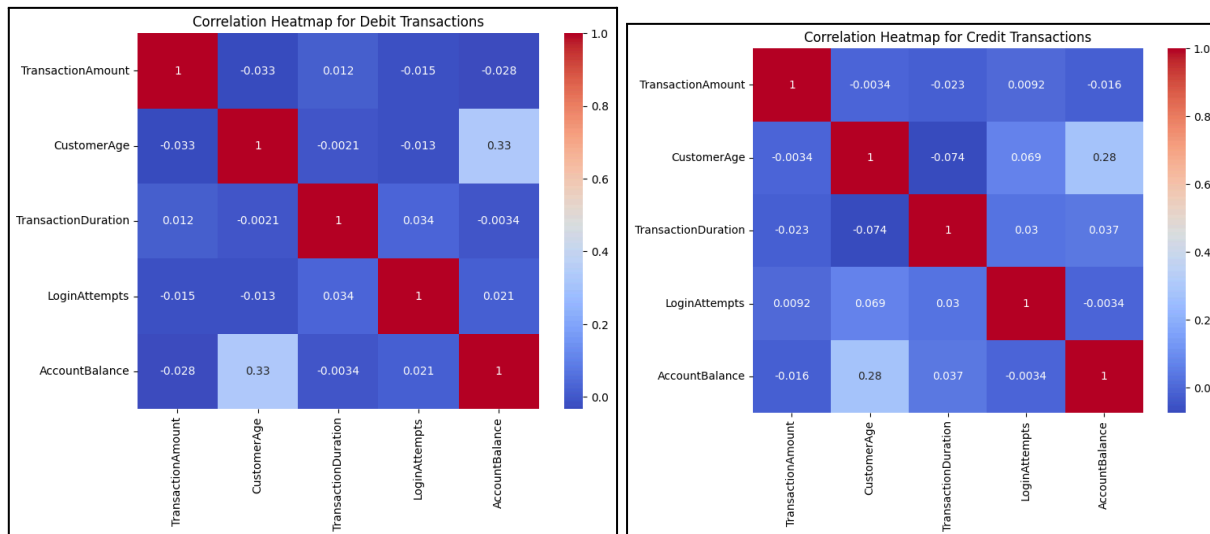
```
    sns.heatmap(  
        subset[numeric_cols].corr(),  
        annot=True,  
        cmap='coolwarm'
```

```
    )
```

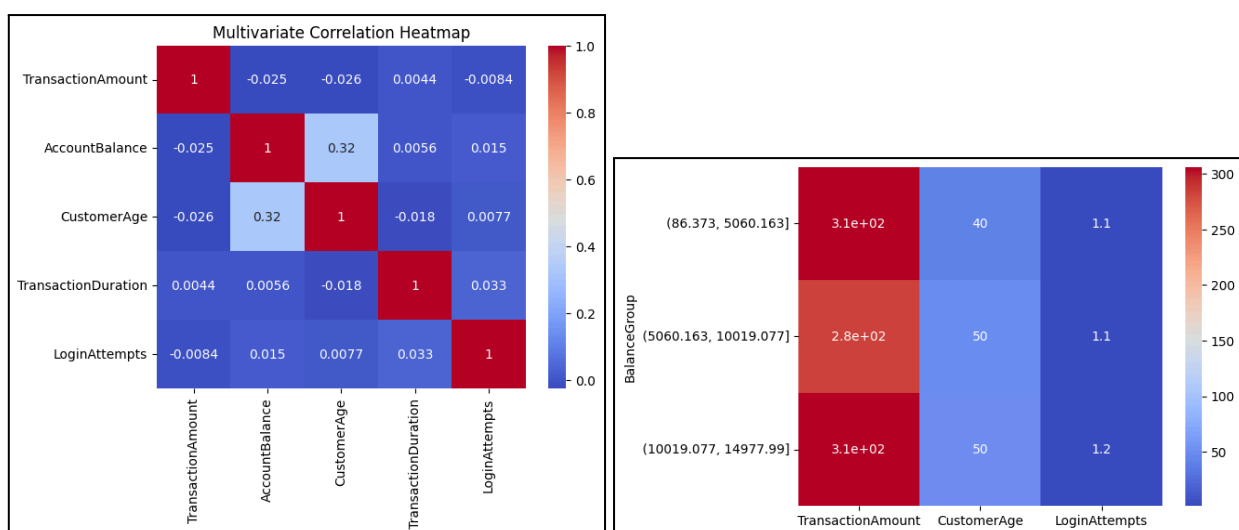
```
    plt.title(f'Correlation Heatmap for {t_type.capitalize()} Transactions')
```

```
    plt.show()
```



```
import seaborn as sns
import matplotlib.pyplot as plt
corr =
df[['TransactionAmount','AccountBalance','CustomerAge','TransactionDuration','LoginAttempts']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Multivariate Correlation Heatmap")
plt.show()
df['BalanceGroup'] = pd.cut(df['AccountBalance'], bins=3)
group_corr =
df.groupby('BalanceGroup')[['TransactionAmount','CustomerAge','LoginAttempts']].mean()
sns.heatmap(group_corr, annot=True, cmap='coolwarm')
plt.figure(figsize=(8,6))
```



```
data['TransactionDate'] = pd.to_datetime(data['TransactionDate'])
data['PreviousTransactionDate'] = pd.to_datetime(data['PreviousTransactionDate'])
data['TimeSinceLastTransaction'] = (
    data['TransactionDate'] - data['PreviousTransactionDate']
```

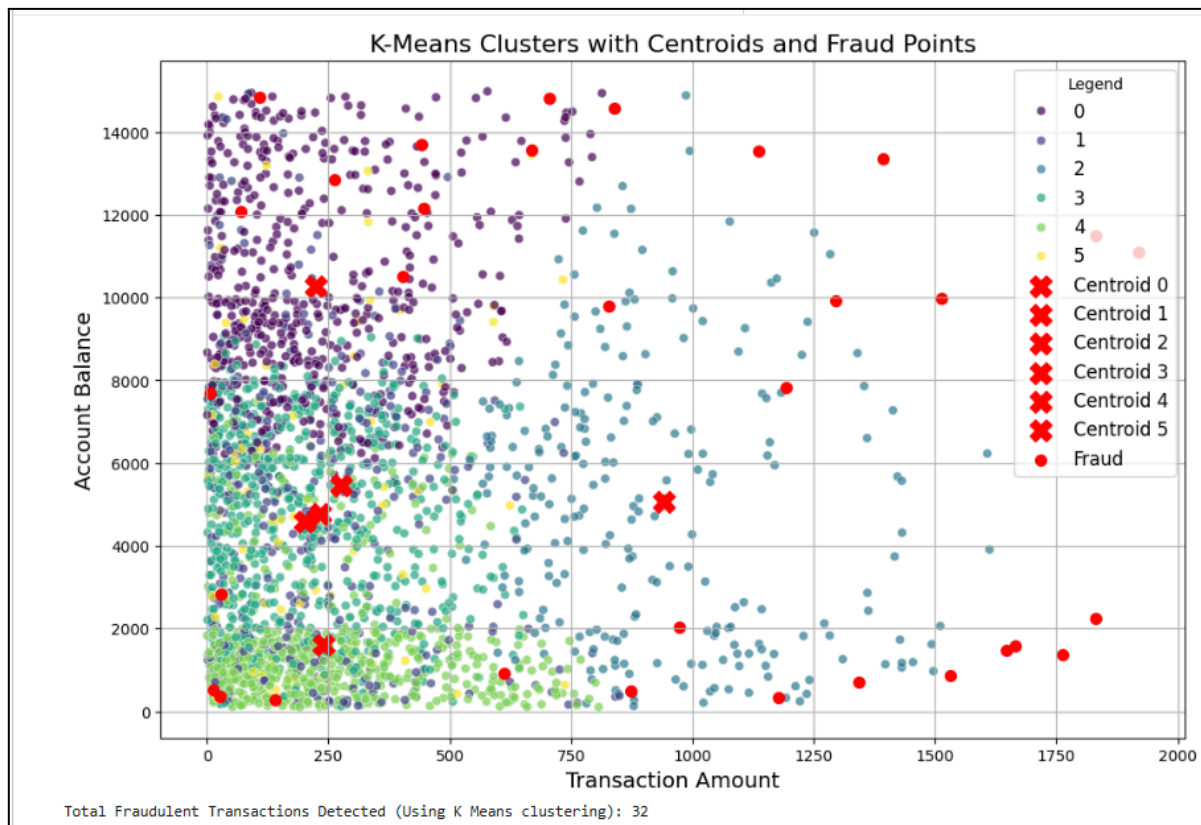
Subject: Python for Data Science(BTCO13602)
OEP Title: Banking Fraud Detection System
Members:.,Rudri Desai, Krisha Dhola, Diya Patel, Kavya Patel, Masoom Parekh
Enrollment No:
ET23BTCO008,ET23BTCO009,E23BTCO010,ET23BTCO016,ET23BTCO031

).dt.total_seconds()

K means clustering:

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# Identify numeric columns
numeric_cols = ['TransactionAmount', 'TransactionDuration', 'LoginAttempts', 'AccountBalance',
'CustomerAge']
# Standardize numeric columns for clustering
scaler = StandardScaler()
numeric_scaled = pd.DataFrame(scaler.fit_transform(data[numeric_cols]), columns=numeric_cols)
# Perform K-Means clustering
kmeans = KMeans(n_init='auto') # FutureProofing
kmeans.fit(numeric_scaled)
n_clusters = 5 # Adjust the number of clusters as needed
kmeans = KMeans(n_clusters=n_clusters, random_state=42)
kmeans_labels = kmeans.fit_predict(numeric_scaled)
data['KMeans_Cluster'] = kmeans_labels
# Calculate distances from centroids
centroids = kmeans.cluster_centers_
distances = np.linalg.norm(numeric_scaled.values - centroids[kmeans_labels], axis=1)
data['KMeans_Distance'] = distances
# Set threshold: Mean + 3 * Standard Deviation
threshold = distances.mean() + 3 * distances.std()
data['KMeans_Fraud'] = distances > threshold
# Summary of flagged anomalies
fraud_summary = data['KMeans_Fraud'].value_counts()
threshold, fraud_summary, data[['TransactionAmount', 'KMeans_Distance',
'KMeans_Fraud']].head()
# Calculate centroids and scale them back to the original feature space
centroids_scaled = kmeans.cluster_centers_
centroids_original = scaler.inverse_transform(centroids_scaled)
# Add a fraud detection logic (using distance from centroids)
distances = np.linalg.norm(numeric_scaled - centroids_scaled[kmeans_labels], axis=1)
threshold = distances.mean() + 3 * distances.std()
data['Fraud'] = distances > threshold
data['KMeans_Distance'] = distances
# Scatterplot for two features with clusters
plt.figure(figsize=(12, 8))
sns.scatterplot(
    x=data['TransactionAmount'],
    y=data['AccountBalance'],
    hue=data['KMeans_Cluster'],
    palette='viridis',
```

```
alpha=0.7,
legend='full'
)
# Overlay centroids on the scatterplot
for i, centroid in enumerate(centroids_original):
    plt.scatter(
        centroid[numeric_cols.index('TransactionAmount')],
        centroid[numeric_cols.index('AccountBalance')],
        color='red',
        marker='X',
        s=200,
        label=f'Centroid {i}'
    )
# Highlight fraud points
fraud_points = data[data['Fraud']]
plt.scatter(
    fraud_points['TransactionAmount'],
    fraud_points['AccountBalance'],
    color='red',
    marker='o',
    s=50,
    label='Fraud'
)
plt.title('K-Means Clusters with Centroids and Fraud Points', fontsize=16)
plt.xlabel('Transaction Amount', fontsize=14)
plt.ylabel('Account Balance', fontsize=14)
plt.legend(title='Legend', fontsize=12)
plt.grid(True)
plt.show()
# Print summary of fraud points
total_fraud_points = data['Fraud'].sum()
print(f"Total Fraudulent Transactions Detected (Using K Means clustering): {total_fraud_points}")
```



Z score analysis:

```
from scipy.stats import zscore
```

```
# Define numeric columns
```

```
numeric_cols = ['TransactionAmount', 'TransactionDuration', 'LoginAttempts', 'AccountBalance',  
'CustomerAge']
```

```
# Calculate Z-Scores
```

```
z_scores = np.abs(zscore(data[numeric_cols]))
```

```
data['ZScore_Fraud'] = (z_scores > 3).any(axis=1) # Flag transactions where any feature exceeds  
|z| > 3
```

```
# Extract fraudulent transactions
```

```
zscore_fraud_points = data[data['ZScore_Fraud']]
```

```
# Summary of results
```

```
total_zscore_fraud_points = data['ZScore_Fraud'].sum()
```

```
zscore_fraud_points_summary = zscore_fraud_points[numeric_cols + ['ZScore_Fraud']]
```

```
# Adding True values to Fraud column
```

```
data['Fraud'] |= data['ZScore_Fraud']
```

```
# Scatterplot for Z-Score fraud detection
```

```
plt.figure(figsize=(12, 8))
```

```
sns.scatterplot(
```

```
    x=data['TransactionAmount'],
```

```
    y=data['AccountBalance'],
```

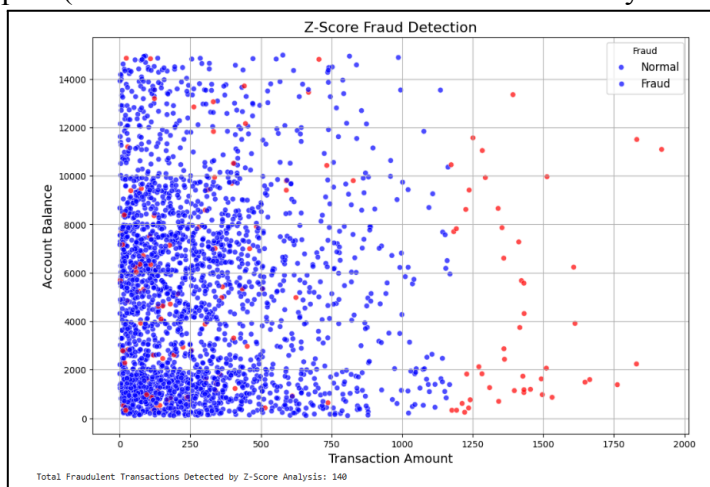
```
    hue=data['ZScore_Fraud'],
```

```
    palette={True: 'red', False: 'blue'},
```

```
    alpha=0.7
```

```
)
```

```
plt.title('Z-Score Fraud Detection', fontsize=16)
plt.xlabel('Transaction Amount', fontsize=14)
plt.ylabel('Account Balance', fontsize=14)
plt.legend(title='Fraud', labels=['Normal', 'Fraud'], fontsize=12)
plt.grid(True)
plt.show()
# Save fraudulent transactions detected by Z-Score analysis
zscore_fraud_output_path = 'bank_transactions_data_2.csv'
zscore_fraud_points.to_csv(zscore_fraud_output_path, index=False)
# Summary
print(f'Total Fraudulent Transactions Detected by Z-Score Analysis: {total_zscore_fraud_points}')
```

**Logistic Regression:**

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
# Define relevant columns for logistic regression
features = ['TransactionAmount', 'TransactionDuration', 'LoginAttempts',
'AccountBalance', 'CustomerAge', 'TimeSinceLastTransaction']
target = 'Fraud'
# Prepare the dataset
X = data[features]
y = data[target]
# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=42)
# Train Logistic Regression model
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)
# Predict fraud on the test set
y_pred = log_reg.predict(X_test)
# Evaluate model performance
print("Classification Report:")
```

```
print(classification_report(y_test, y_pred))
# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", xticklabels=['Non-Fraud', 'Fraud'],
yticklabels=['Non-Fraud', 'Fraud'])
plt.title('Confusion Matrix', fontsize=16)
plt.xlabel('Predicted', fontsize=14)
plt.ylabel('Actual', fontsize=14)
plt.show()
# Add predictions to the dataset
data['LogReg_Fraud'] = log_reg.predict(X_scaled)
data['Fraud'] |= data['LogReg_Fraud']
# Visualize fraud vs. non-fraud transactions
plt.figure(figsize=(12, 8))
sns.scatterplot(
    x=data['TransactionAmount'],
    y=data['AccountBalance'],
    hue=data['LogReg_Fraud'],
    palette={1: 'red', 0: 'blue'},
    alpha=0.7
)
plt.title('Logistic Regression Fraud Detection', fontsize=16)
plt.xlabel('Transaction Amount', fontsize=14)
plt.ylabel('Account Balance', fontsize=14)
plt.legend(title='Fraud', labels=['Non-Fraud', 'Fraud'], fontsize=12)
plt.grid(True)
plt.show()
# Save fraudulent transactions detected by Logistic Regression
log_reg_fraud_output_path = 'bank_transactions_data_2.csv'
log_reg_fraud_points = data[data['LogReg_Fraud'] == 1]
log_reg_fraud_points.to_csv(log_reg_fraud_output_path, index=False)
# Summary
total_log_reg_fraud_points = log_reg_fraud_points.shape[0]
print(f"Total Fraudulent Transactions Detected by Logistic Regression:
{total_log_reg_fraud_points}")
print(f"Fraudulent transactions saved to: {log_reg_fraud_output_path}")
```

Classification Report:				
	precision	recall	f1-score	support
False	0.99	1.00	1.00	714
True	1.00	0.85	0.92	40
accuracy			0.99	754
macro avg	1.00	0.93	0.96	754
weighted avg	0.99	0.99	0.99	754

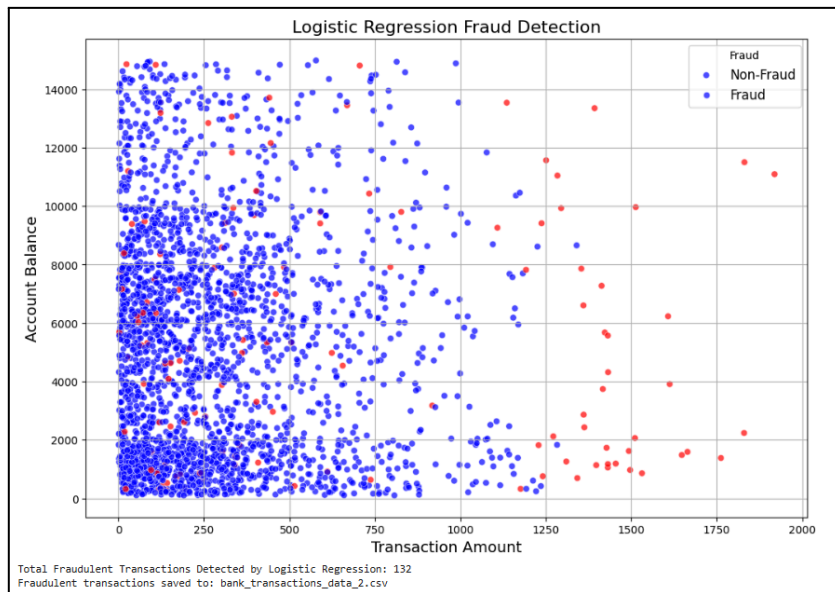
Subject: Python for Data Science(BTCO13602)

OEP Title: Banking Fraud Detection System

Members: Rudri Desai, Krisha Dhola, Diya Patel, Kavya Patel, Masoom Parekh

Enrollment No:

ET23BTCO008,ET23BTCO009,E23BTCO010,ET23BTCO016,ET23BTCO031



Fraud data:

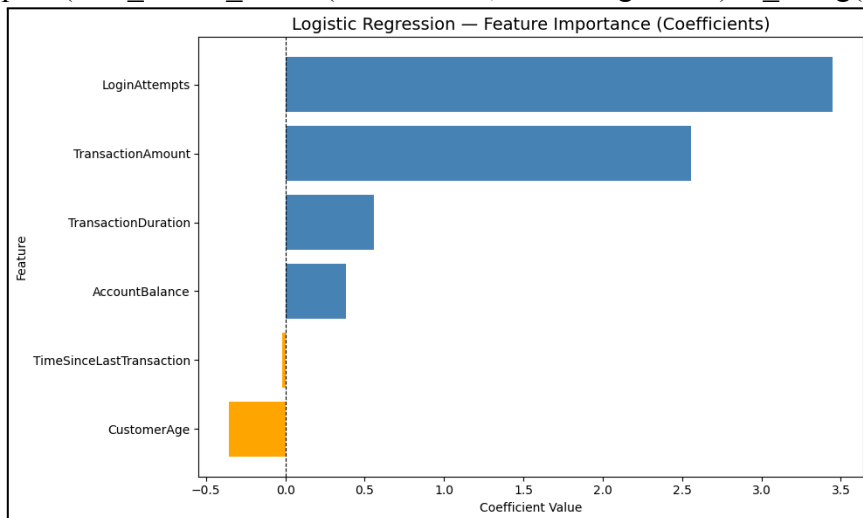
```
# Save FULL dataset with all fraud flags
data.to_csv('bank_transactions_final.csv', index=False)
print("Dataset saved with Fraud column")
data[data['Fraud'] == True]
```

Dataset saved with fraud column																			
TransactionID	AccountID	TransactionAmount	TransactionDate	TransactionType	Location	DeviceID	IP Address	MerchantID	Channel	...	LoginAttempts	AccountBalance	PreviousTransactionDate	KMeans_Cluster	KMeans_Distance	KMeans_Fraud	Fraud	TimeSinceLastTransaction	Score_Fraud
22	TX000024	AC000453	345.84	2023-05-02 16:35:46	Debit	Fort Worth	0000162	191.83.103.186	M063	Online	...	3	1402.50	2024-11-04 08:07:04	1	2.361804	False	True	-47655678.0
26	TX000027	AC000441	346.83	2023-04-17 16:37:01	Debit	Miami	0000046	55.154.161.250	M028	ATM	...	5	873.35	2024-11-04 08:11:38	1	2.412339	False	True	-48858477.0
32	TX000033	AC000060	386.45	2023-09-25 16:36:00	Debit	New York	0000821	133.67.250.163	M067	ATM	...	3	9080.15	2024-11-04 08:11:13	1	2.116263	False	True	-35048713.0
38	TX000038	AC000478	785.31	2023-05-13 17:07:40	Debit	Mexico	0000077	49.38.37.185	M016	ATM	...	2	7914.68	2024-11-04 08:11:17	2	2.341088	False	True	-33077417.0
74	TX000075	AC000065	1312.31	2023-05-04 16:36:28	Debit	Indianapolis	0000331	163.83.8.182	M036	Branch	...	1	605.86	2024-11-04 08:06:51	2	2.438385	False	True	-34370323.0
...																			
2403	TX000404	AC000111	1483.06	2023-05-07 17:05:41	Debit	Colorado Springs	0000344	136.162.111.135	M056	ATM	...	1	1618.16	2024-11-04 08:07:07	2	2.531323	False	True	-44550086.0
2414	TX000415	AC000038	1664.33	2023-09-25 17:11:19	Debit	San Antonio	0000073	116.106.307.136	M054	Branch	...	1	1588.31	2024-11-04 08:07:07	2	3.161577	True	True	-35045748.0
2423	TX000424	AC000340	332.06	2023-05-21 16:38:54	Debit	Seattle	0000365	84.37.250.136	M056	ATM	...	4	11630.52	2024-11-04 08:06:33	1	3.355094	False	True	-43343018.0
2445	TX000446	AC000436	403.01	2023-09-04 17:32:35	Debit	Washington	0000677	225.32.70.156	M039	Online	...	3	10513.52	2024-11-04 08:06:32	1	3.357355	True	True	-36059017.0
2452	TX000456	AC000043	655.81	2023-05-05 16:32:41	Debit	Covina	0000507	86.82.81.143	M014	ATM	...	2	4238.88	2024-11-04 08:12:08	2	2.961405	False	True	-47462336.0

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
# Get feature names (numeric + encoded categorical if used)
feature_names = ['TransactionAmount', 'TransactionDuration',
                  'LoginAttempts', 'AccountBalance',
                  'CustomerAge', 'TimeSinceLastTransaction']
# Extract coefficients from trained logistic regression model
coefficients = log_reg.coef_[0]
# Create dataframe for visualization
coef_df = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': coefficients
}).sort_values('Coefficient', ascending=True)
# Plot
plt.figure(figsize=(10, 6))
colors = ['steelblue' if c > 0 else 'orange' for c in coef_df['Coefficient']]
plt.barh(coef_df['Feature'], coef_df['Coefficient'], color=colors)
plt.axvline(x=0, color='black', linewidth=0.8, linestyle='--')
```



```
plt.title('Logistic Regression — Feature Importance (Coefficients)', fontsize=14)
plt.xlabel('Coefficient Value')
plt.ylabel('Feature')
plt.tight_layout()
plt.show()
# Print ranked importance
print("Feature Importance Ranking:")
print(coef_df.sort_values('Coefficient', ascending=False).to_string(index=False))
```



Feature Importance Ranking:	
Feature	Coefficient
LoginAttempts	3.450553
TransactionAmount	2.558498
TransactionDuration	0.559456
AccountBalance	0.382506
TimeSinceLastTransaction	-0.017922
CustomerAge	-0.355779

Feature Importance:

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = LogisticRegression()
model.fit(X_scaled, y)
importance = model.coef_[0]
feat_imp = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importance
}).sort_values(by='Importance', ascending=False)
print(feat_imp)
```

	Feature	Importance
2	LoginAttempts	4.393572
0	TransactionAmount	3.082404
1	TransactionDuration	0.584478
3	AccountBalance	0.302828
5	TimeSinceLastTransaction	0.167020
4	CustomerAge	-0.322898

Analysis of new data:

```
new_data = pd.DataFrame([
    {
        'TransactionAmount': 50000,
        'TransactionDuration': 120,
        'LoginAttempts': 3,
        'AccountBalance': 2000,
        'CustomerAge': 45,
        'TimeSinceLastTransaction': 3600
    },
    {
        'TransactionAmount': 200,
        'TransactionDuration': 20,
        'LoginAttempts': 1,
        'AccountBalance': 50000,
        'CustomerAge': 30,
        'TimeSinceLastTransaction': 100
    },
    {
        'TransactionAmount': 90000,
        'TransactionDuration': 300,
        'LoginAttempts': 6,
        'AccountBalance': 1000,
        'CustomerAge': 50,
        'TimeSinceLastTransaction': 50
    },
    {'TransactionAmount': 75000, 'TransactionDuration': 250, 'LoginAttempts': 4, 'AccountBalance': 1500, 'CustomerAge': 40, 'TimeSinceLastTransaction': 200},
    {'TransactionAmount': 100, 'TransactionDuration': 10, 'LoginAttempts': 1, 'AccountBalance': 80000, 'CustomerAge': 25, 'TimeSinceLastTransaction': 5000},
    {'TransactionAmount': 60000, 'TransactionDuration': 180, 'LoginAttempts': 5, 'AccountBalance': 3000, 'CustomerAge': 35, 'TimeSinceLastTransaction': 60},
    {'TransactionAmount': 15000, 'TransactionDuration': 60, 'LoginAttempts': 2, 'AccountBalance': 12000, 'CustomerAge': 28, 'TimeSinceLastTransaction': 800},
    {'TransactionAmount': 82000, 'TransactionDuration': 240, 'LoginAttempts': 5, 'AccountBalance': 1800, 'CustomerAge': 47, 'TimeSinceLastTransaction': 90},
    {'TransactionAmount': 300, 'TransactionDuration': 15, 'LoginAttempts': 1, 'AccountBalance': 60000, 'CustomerAge': 22, 'TimeSinceLastTransaction': 5000},
    {'TransactionAmount': 67000, 'TransactionDuration': 200, 'LoginAttempts': 4, 'AccountBalance': 2500, 'CustomerAge': 39, 'TimeSinceLastTransaction': 150},
    {'TransactionAmount': 1200, 'TransactionDuration': 30, 'LoginAttempts': 2, 'AccountBalance': 40000, 'CustomerAge': 33, 'TimeSinceLastTransaction': 2000},
    {'TransactionAmount': 95000, 'TransactionDuration': 320, 'LoginAttempts': 7, 'AccountBalance':
```

Subject: Python for Data Science(BTCO13602)
OEP Title: Banking Fraud Detection System
Members: Rudri Desai, Krisha Dhola, Diya Patel, Kavya Patel, Masoom Parekh
Enrollment No:
ET23BTCO008,ET23BTCO009,E23BTCO010,ET23BTCO016,ET23BTCO031

```
900, 'CustomerAge': 52, 'TimeSinceLastTransaction': 40},
    {'TransactionAmount': 45000, 'TransactionDuration': 150, 'LoginAttempts': 3, 'AccountBalance':
5000, 'CustomerAge': 36, 'TimeSinceLastTransaction': 300},
    {'TransactionAmount': 700, 'TransactionDuration': 20, 'LoginAttempts': 1, 'AccountBalance':
70000, 'CustomerAge': 26, 'TimeSinceLastTransaction': 7000},
    {'TransactionAmount': 88000, 'TransactionDuration': 280, 'LoginAttempts': 6, 'AccountBalance':
1100, 'CustomerAge': 49, 'TimeSinceLastTransaction': 70},
    {'TransactionAmount': 2500, 'TransactionDuration': 40, 'LoginAttempts': 2, 'AccountBalance':
35000, 'CustomerAge': 31, 'TimeSinceLastTransaction': 1200}
])
features = ['TransactionAmount', 'TransactionDuration',
            'LoginAttempts', 'AccountBalance',
            'CustomerAge', 'TimeSinceLastTransaction']
new_data = new_data[features]
new_scaled = scaler.transform(new_data)
predictions = log_reg.predict(new_scaled)
probabilities = log_reg.predict_proba(new_scaled)[: , 1]
results = new_data.copy()
```

```
results['Fraud_Prediction'] = predictions # 0 = Normal, 1 = Fraud
results['Fraud_Probability'] = probabilities
```

Results :

	TransactionAmount	TransactionDuration	LoginAttempts	AccountBalance	CustomerAge	TimeSinceLastTransaction	Fraud_Prediction	Fraud_Probability
0	50000	120	3	2000	45	3600	True	1.000000
1	200	20	1	50000	30	100	False	0.004467
2	90000	300	6	1000	50	50	True	1.000000
3	75000	250	4	1500	40	200	True	1.000000
4	100	10	1	80000	25	5000	False	0.034861
5	60000	180	5	3000	35	60	True	1.000000
6	15000	60	2	12000	28	800	True	1.000000
7	82000	240	5	1800	47	90	True	1.000000
8	300	15	1	60000	22	5000	False	0.031391
9	67000	200	4	2500	39	150	True	1.000000
10	1200	30	2	40000	33	2000	True	0.999704
11	95000	320	7	900	52	40	True	1.000000
12	45000	150	3	5000	36	300	True	1.000000
13	700	20	1	70000	26	7000	True	0.734490
14	88000	280	6	1100	49	70	True	1.000000
15	2500	40	2	35000	31	1200	True	1.000000